LAB 4
Implementation of the game PONG

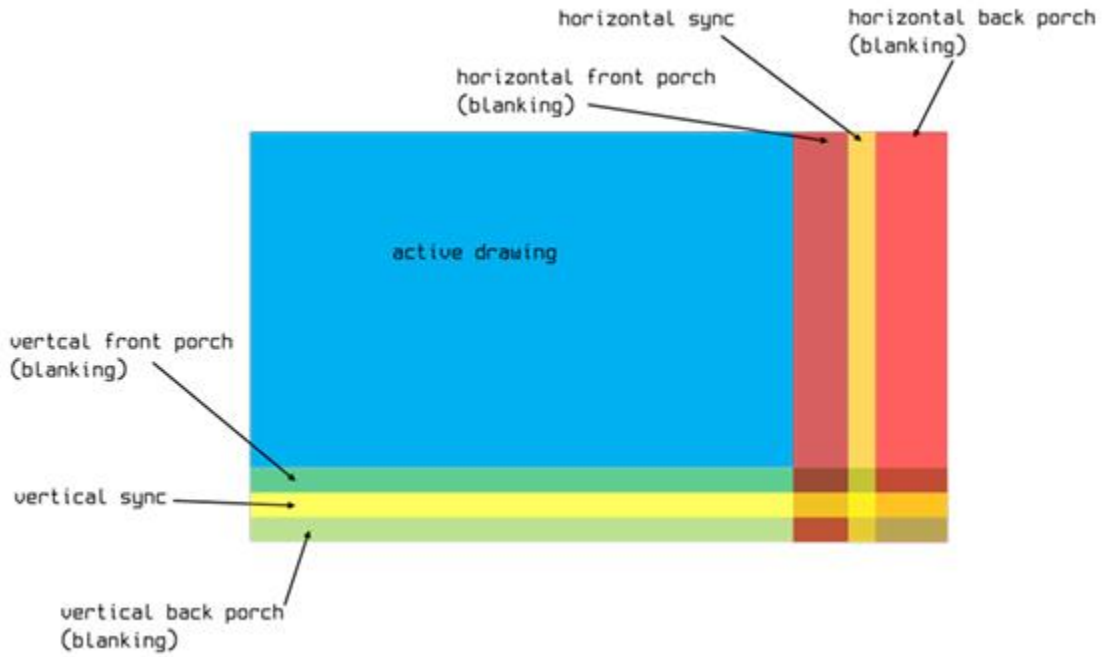
Name: Ioannis Papadopoulos, Maria Ilektra Tzormpatzaki
Student ID: 03899, 03993
Date: 15-1-2026
Board: Boolean Board (Spartan-7 FPGA)

Contents

1. Introduction

This project focuses on the implementation of the classic Pong game using the HDMI video controller developed in Lab 3.

For verification purposes, reduced screen timing parameters are used instead of the standard 640×480 resolution, allowing easier observation and debugging of the game behavior.



Active Drawing (H)	10
Horizontal Front Porch	2
Horizontal Sync	5
Horizontal Back Porch	3
Total Horizontal	20
Active Drawing (V)	8
Vertical Front Porch	1
Vertical Sync	3
Vertical Back Porch	2
Total Vertical	14

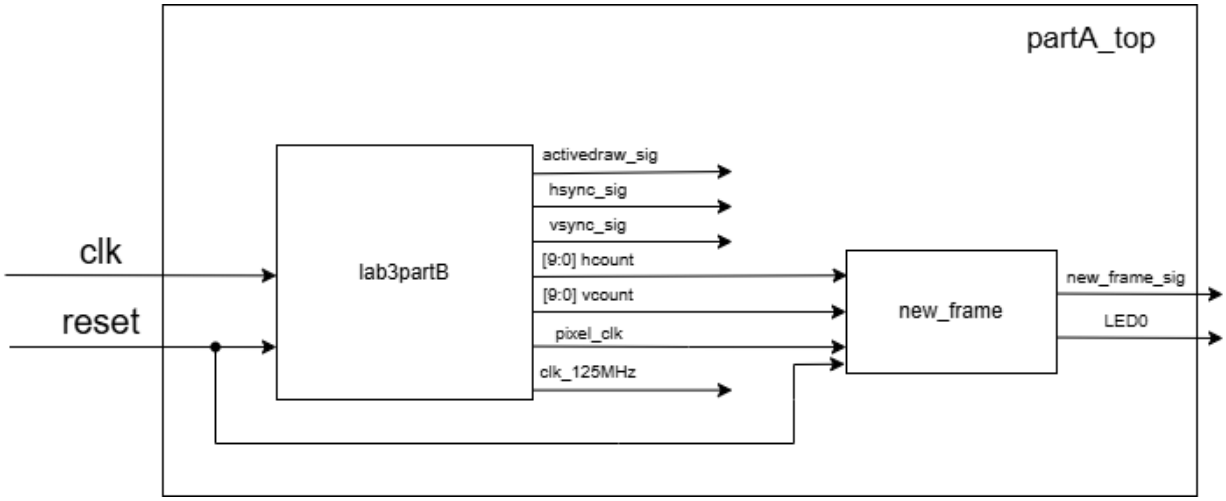
Also, the game objects are also scaled accordingly:

- Puck= 2×2 pixel object
- Paddle = 2×4 pixel object

2. Part A – Generation and Verification of the new_frame signal

2.1 Introduction

In this part, a new_frame signal is generated to indicate when a complete video frame has finished, and a new one is about to be displayed. This signal is later used to synchronize the game logic, while an on-board LED provides visual confirmation that 60 frames have passed.



2.2 Implementation

The new_frame signal is asserted for exactly one pixel clock cycle when the video timing counters reach the last pixel of the active drawing region, specifically at hcount = 639 and vcount = 479. To keep track when 60 frames pass a simple counter increments on each new_frame event and toggles the LED, corresponding to approximately one second at a 60 Hz frame rate.

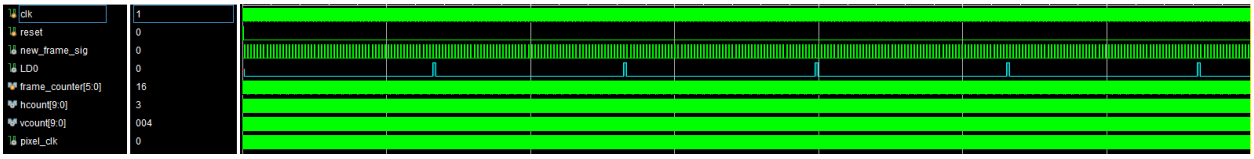
2.3 Verification

The testbench is very simple, it just instantiates the top module and runs until multiple new frames are passed.

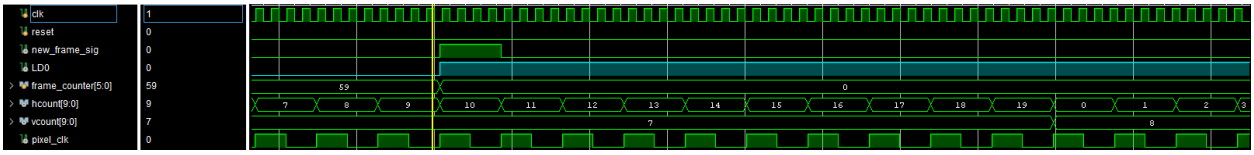
In the full waveform 5 instances of the LD0 going high can be observed, meaning that more than 300 frames have passed.

The zoomed in waveform shows that the LD0 correctly lights at the end of 60th new_frame. In the screenshot its after the frame counter passes 59 because it starts counting from 0. The newframe changes correctly when hcount = 9 and vcount = 7, keep in mind that the sizing of each frame is smaller for the simulation as explained in the ‘‘1. Introduction’’.

1. Full waveform:



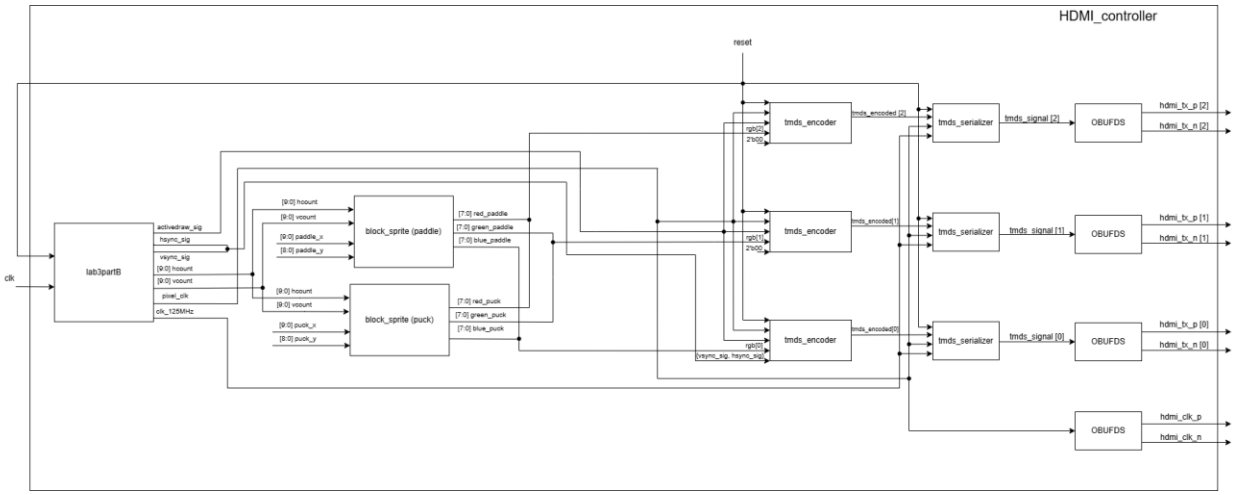
2. Zoomed in waveform:



3. Part B – Visualization of Block Sprites

3.1 Introduction

In this part, we transition from the static VRAM-based image of the previous lab to a dynamic object-based rendering system. The objective is to design a hardware module capable of determining in real-time if the current pixel being scanned by the HDMI controller belongs to a specific game object (sprite) or the background based on predefined coordinates and dimensions.



3.2 Implementation

The important part of the implementation relies on two main components: the block_sprite module and the updated top module HDMI_controller

- **Block Sprite Module:**

  The block_sprite is a purely combinational circuit designed to render rectangular objects on the screen. It takes the current pixel coordinates (hcount, vcount) and the top-left corner position of the object (sprite_x, sprite_y) as inputs. Using parameters for SPRITE_WIDTH and SPRITE_HEIGHT, the module performs a range check: if the current pixel falls within

the boundaries [sprite_x, sprite_x + SPRITE_WIDTH] and [sprite_y, sprite_y + SPRITE_HEIGHT], it outputs the object's color (white). Otherwise, it outputs the background color (black). This approach allows for the creation of any rectangular game element, in this case the puck and the paddle.

- **HDMI Controller top module:**

  The HDMI_controller was modified to act as the top-level orchestrator for these sprites. Instead of fetching data from the VRAM, it now instantiates two block_sprite modules one for the puck and one for the paddle. The RGB outputs from these modules are combined using bitwise OR logic to create the final video signal. In this stage, the position inputs for the sprites stay constant to verify their correct rendering at fixed locations.

3.3 Verification

For the simulation the sizing of each frame is as explained in the “1. Introduction”. Also, the top left corner of the objects is:

- Puck: puck_x = 5 and puck_y = 2.
- Paddle: paddle_x = 2 and paddle_y = 2.

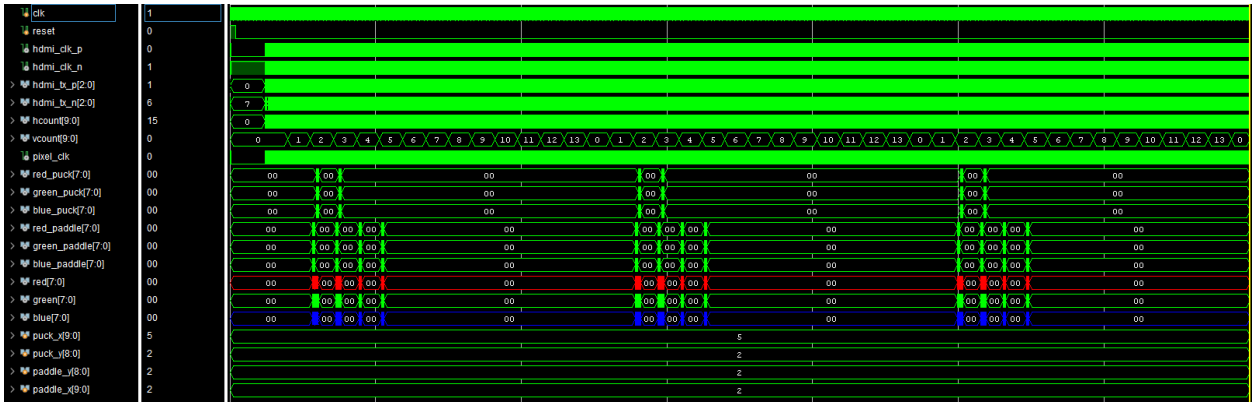
The Puck size is 2x2, while the Paddle size is 2x4, as explained in the “1. Introduction”

The Full waveform is around the frames long. The rgb of the puck and the paddle appear at different points of each frame and the background is correctly mostly black (00).

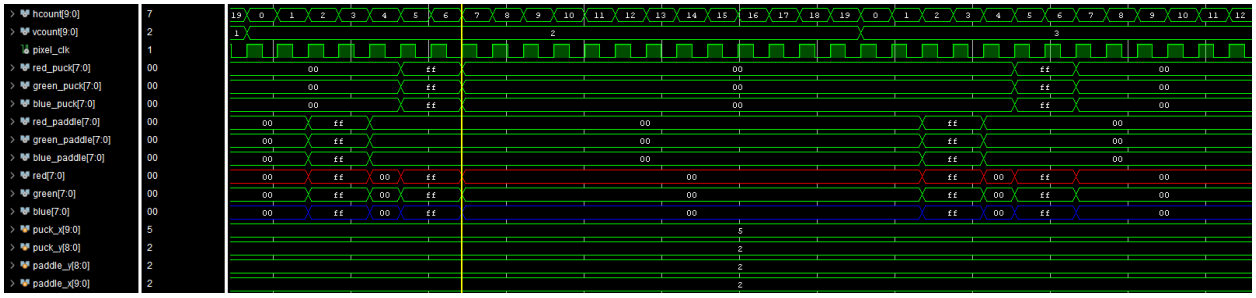
In the zoomed in waveforms the rgb of the puck and paddle can easily be tracked. The rgb of the puck is white (FF) when hcount = 5,6 and vcount = 2,3 because it covers an area of 2x2 pixels and starts at the [5,2] pixel. With the same logic, the rgb of the paddle is white when hcount = 2,3 and vcount = 2,3,4,5 because it covers an area of 2x4 pixels and starts at the [2,2] pixel.

At every point the actual rgb is the bitwise OR product of the puck and paddle rgb.

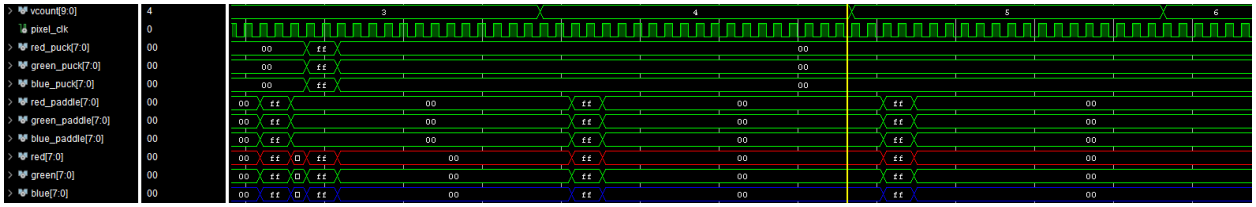
1. Full waveform:



2. 1st Zoomed in waveform:



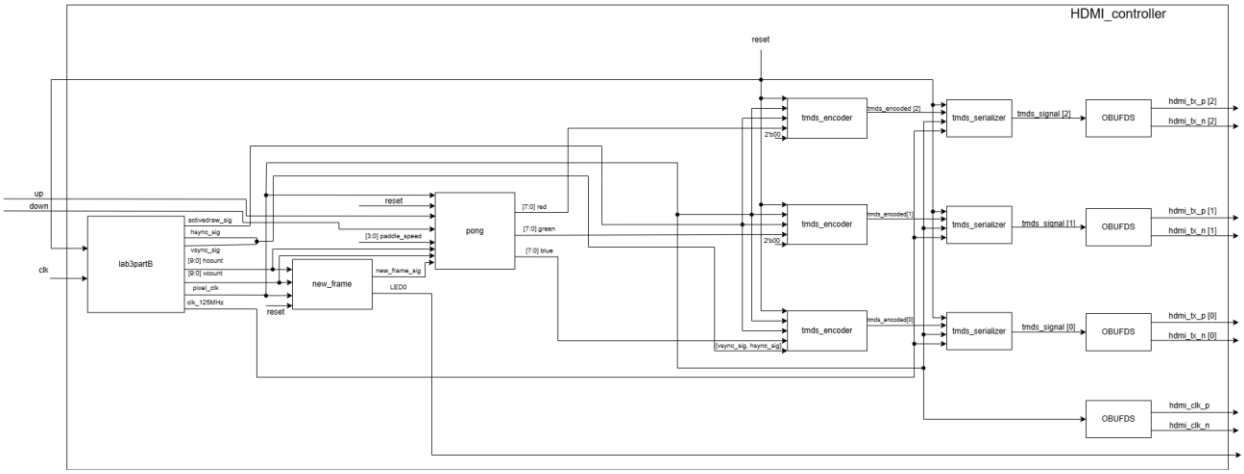
3. 2nd Zoomed in waveform:



4. Part C – Pong Module Structure & Paddle Movement

4.1 Introduction

In this stage, the system architecture is upgraded by creating a central game control unit, the pong module. Also, a user-controlled paddle movement is implemented by updating its vertical position once per frame.



4.2 Implementation

- **Architectural changes**

  The core difference in this part is the centralization of the game logic within the pong.v file.

1. **Pong Instatiations:** The block_sprite modules are no longer instantiated directly within the HDMI_controller but are instead placed inside the pong module.
2. **Color Rendering:** The processing of colors and the bitwise OR logic (used to combine the puck and paddle sprites) is performed internally within the pong module.
3. **Top-Level Architecture:** The HDMI_controller instantiating only the pong unit and passing the necessary timing signals (hcount, vcount, new_frame) and control inputs to it, along with instantiating the rest of the modules from Lab3.

- **Paddle movement logic**

  The paddle movement is implemented as a sequential circuit that responds to user button inputs (up, down) and the speed defined by the switches (paddle_speed).

1. **Frame Synchronization:** The update of the paddle's vertical position (paddle_y) occurs exclusively when the new_frame signal (from Part A) is active. This ensures the paddle moves only once per frame, maintaining a consistent 60Hz update
2. **Movement Controls:** Using the on-board buttons (BTN2 and BTN3), the player can move the paddle vertically. The displacement speed is determined in pixels per frame based on the value of SW12–SW15.
3. **Boundary Conditions:** Specific limit-checking logic is implemented to prevent the paddle from exiting the visible screen area (0 to 480 pixels). Even at high speeds, the system checks if the next position would exceed the screen boundaries and "locks" the paddle at the edge.

4.3 Verification

The frame and sprites sizing is the same as before. Also the starting position of the paddle and the puck is the same as in PartB Verification

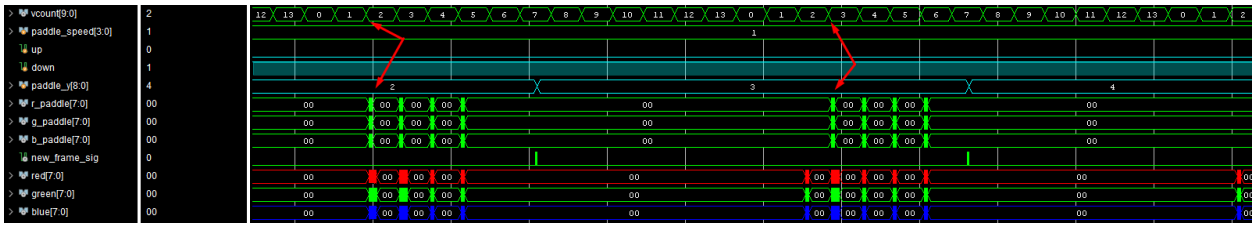
The Full Waveform shows:

- Firstly, the paddle stays still at paddle_y = 2
- Then the down button is “pressed” while the paddle_speed = 1, meaning the paddle is starting to move down one pixel per frame (every pulse of new_frame_signal). When the top part of the paddle reaches the 4^{th} row, it stops even though the down button is still active. This happens because the active frame area consists of 8 rows (7^{th} being the last). Therefore, when the top of the paddle reaches the 4^{th} row the bottom part of the paddle hits the floor wall and the paddle can’t go any further.
- Lastly, after a while the up button is pressed while the paddle_speed = 3. The top of the paddle correctly moves from the 4^{th} row to 1^{st} row in one frame. In the very next frame, it moves just one pixel although the paddle_speed = 3. This happens because the paddle reaches the top wall, and if it moved up 3 pixels again it would exit the screen.

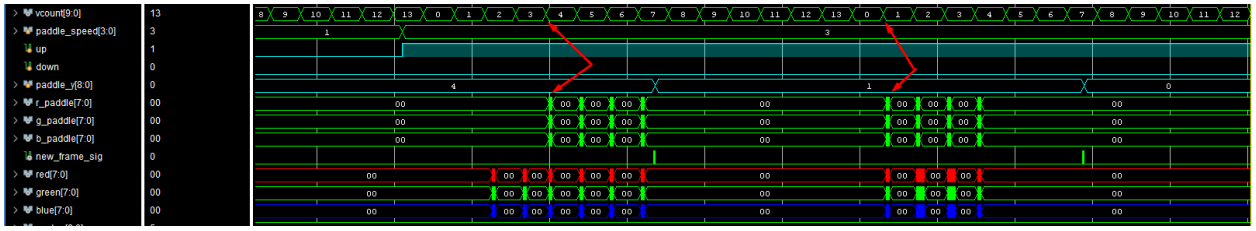
The Zoomed in Waveforms show that when the paddle moves, the rgb also changes accordingly. Proving that the paddle actually moves in the screen.

1. Full waveform:

2. 1st Zoomed in waveform:



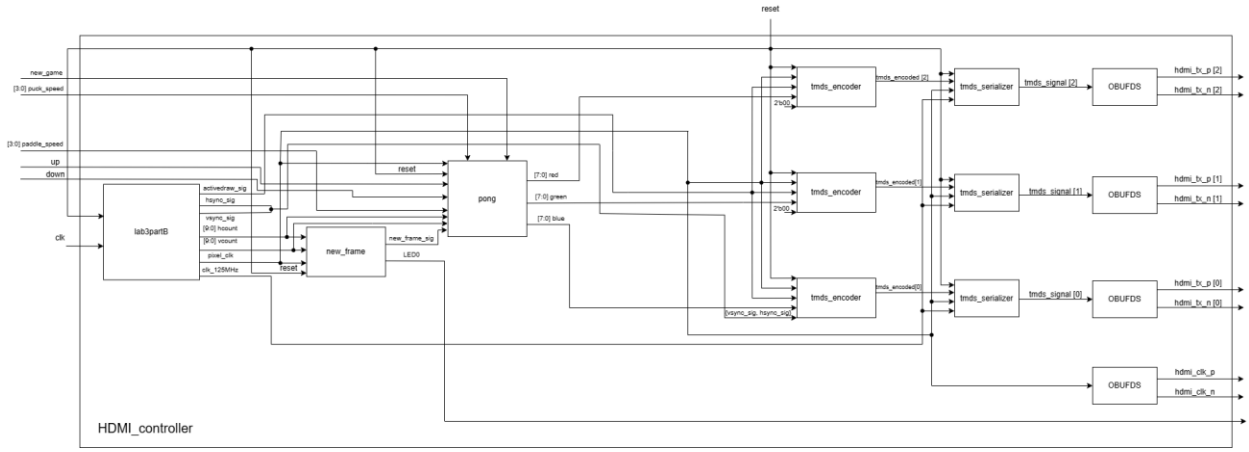
3. 2nd Zoomed in waveform:



5. Part D – Puck Movement, Collision Logic & Game Completion

5.1 Introduction

In this stage, the primary objective is to implement dynamic puck movement and a collision detection system, alongside a "Game Over" state that freezes gameplay upon failure to intercept the puck and a new_game button that begins a game after the previous one ends.



5.2 Implementation

- **Puck Movement Logic**

  The puck's behavior is defined by independent horizontal and vertical movement components, resulting in a continuous diagonal trajectory.

  1. **Frame Synchronization:** Similar to the paddle, all puck position updates for puck_x and puck_y occur only when the new_frame signal is active.
  2. **Vertical Movement (Y-axis):** The puck moves up (y_dir = 0) or down (y_dir = 1) based on the y_dir register. If the puck's next position exceeds the top wall or bottom wall, it is placed at the edge, and the y_dir is toggled to create a bounce effect.
  3. **Horizontal Movement (X-axis):** The puck moves right (x_dir = 1) or left (x_dir = 0) based on the x_dir register. Movement to the right results in a bounce when hitting the right wall. Movement to the left includes specialized logic to detect the paddle face.

- **Collision Detection and Game Over**

  The system must distinguish between a successful paddle deflection and a loss.

  1. **Paddle Collision:** When moving left, the system checks if the puck's X-coordinate reaches the paddle's right edge. If the puck's Y-coordinate is simultaneously within the paddle's vertical span [paddle_y, paddle_y + PADDLE_H], the puck "attaches" to the paddle edge and x_dir is toggled to 1, sending it back to the right. Also to ensure collision detection at higher speeds, the system does not only check for exact coordinate alignment, it also evaluates the puck's projected trajectory. If the puck's current speed would cause it to bypass the paddle's right edge within the next frame, the logic calculates its future vertical position. If this projected Y-coordinate falls within the paddle's vertical span, the puck is intercepted and "attached" precisely to the paddle's outer edge. This predictive approach prevents the puck from visually overlapping with the paddle pixels and ensures that a high-speed collision is not incorrectly flagged as a "Game Over".

2. **Game Over State:** If no collision is detected, the game_over register is asserted. This state disables all further position updates, effectively freezing the game screen.
3. **New Game Initialization:** The new_game signal (controlled by BTN1) allows the player to reset the game. Upon reset, the puck and paddle return to their starting coordinates, and the puck is assigned a pseudo-random starting direction based on the current hcount and vcount LSBs.

- **Top-Level Architecture**

  The HDMI_controller was updated to accommodate these features by adding the new 4-bit switch inputs for puck_speed (SW8-SW11) and the new_game button input (BTN1). These are passed into the pong module.

5.3 Verification

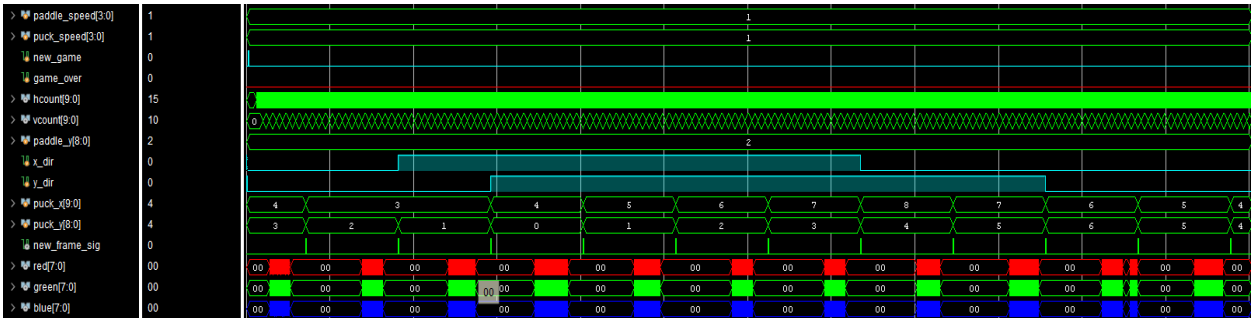
Once again the frame and the sprite sizing is the same. But this time the paddle will be stationary, and only the puck will be moving, to test its collision with the walls and the paddle.

The starting positions of the paddle and the puck as well as the puck speed will be changing in every following simulation to test multiple scenarios. Starting with 2 scenarios were the puck bounces from the paddle and 2 that the puck surpasses the paddle.

The paddle_x is always at , and because of its WIDTH=2, the right face of the paddle is at X = 3.

1. Collision with puck_speed = 1:

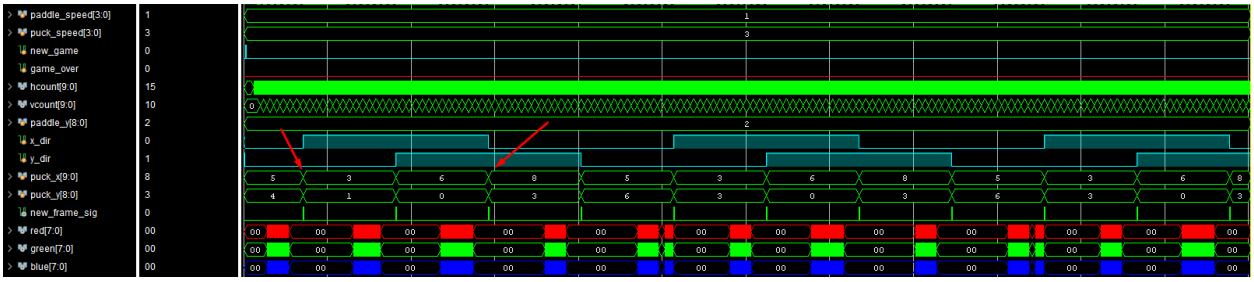
The paddle starts at paddle_y = 2 while the puck starts at puck_x = 4, puck_y = 3 and moves up and left. When it tries to move again on the second frame_signal it realizes that it reached the right face of the paddle. With a puck_y = 2 at that point it collides with the paddle and changes its x directory to the right (x_dir = 1). After a while the puck hits the ceiling, so the y directory also changes. Then at puck_x = 8 the puck hits the right wall because it's 2-pixels wide and the active width of the screen is 10 pixels. Likewise, the puck hits the floor at puck_y = 6 because its 2-pixel lengthy and the active length of the screen is 8 pixels.



All possible collisions were validated with puck_speed =1.

2. Collision with puck_speed = 3:

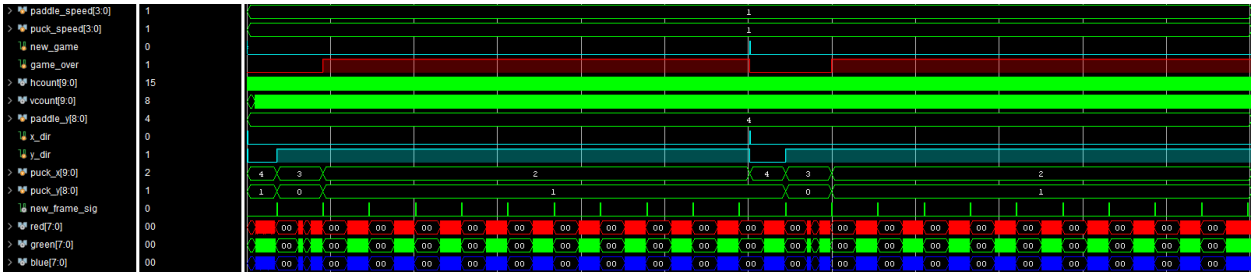
The paddle starts at paddle_y = 2 while the puck starts at puck_x = 5, puck_y = 4 and moves up and left. Now with a higher speed the puck would surpass the right face of the wall immediately at the first new frame, but because as explained in the Implementation above when the puck is about to surpass the paddle we check if the new puck_y to see if the puck would hit the paddle. By doing so the puck collides with the paddle and the puck_x updates to 3(attaching to the paddle) and not 2 (moving passed the paddle). Also by following the red arrows in the waveform below, we can see that the puck collides with the ceiling and the right wall, while also keeping its full body inside the screen



3. Surpassing with puck_speed = 1:

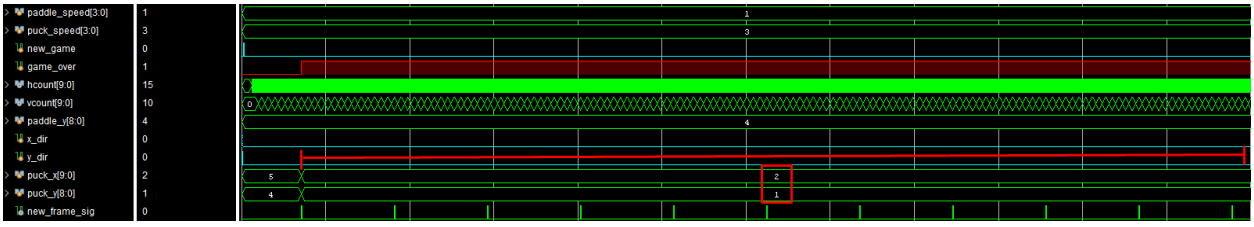
Here the top of the paddle starts paddle_y = 4 while the puck starts at puck_x = 4, puck_y =1 and moves up and left. So, when the new frame the puck moves freely once and collides with the ceiling, but in the next frame it surpasses the paddle. The game_over signal is raised and puck freezes on the screen until the next_game button is pressed.

The next game button is pressed after a while and respawns the puck in the same position, but the puck follows the same directory, so the game eventually ends again. With this test the surpassing of the paddle was checked, along with the gameover and newgame functions.



4. Surpassing with puck_speed = 3:

Even with higher speeds the puck surpasses the paddle and freezes correctly.



6. Part E – BONUS

6.1 Introduction

To enhance the gameplay experience beyond the necessary requirements, several advanced features were integrated. These extensions include a two-player control scheme, a dynamic "Winner" screen utilizing 2 Video RAMs, and a real-time scoring system implemented on the FPGA's 7-segment display.

6.2 Implementation

NOTE: In the Bonus part we were asked to explain the code more thoroughly.

- **Multiplayer Mode (1v1):**

The game was expanded to support a second player by adding a paddle to the right side of the screen. Due to the exhaustion of physical buttons, the right paddle is controlled via on-board switches (SW0 and SW1).

The only change to the code to make this happen was in the “pong.v” file. Firstly, another block sprite instantiation was added for the right paddle. Also, the collision logic was updated, previously when the puck was moving to the right the only collision to check for the x axis was to see if the right side of the puck reached the right wall. Now, the logic was essentially copied from the left side where the puck checks if it will collide with the paddle, and the right side uses the same ideas.

```
// --- Horizontal Puck Movement & Collision ---
if (x_dir) begin // Moving Right
    if (puck_x + PUCK_W + puck_speed <= PADDLE_X_RIGHT)
        puck_x <= puck_x + puck_speed;
    else if (((puck_x + PUCK_W == PADDLE_X_RIGHT) && ((puck_y + PUCK_H >= paddle_y_right) && (puck_y <= paddle_y_right + PADDLE_H))) ||
             (((puck_x + PUCK_W != PADDLE_X_RIGHT) && y_dir == 0) && ((puck_y - puck_speed + PUCK_H >= paddle_y_right) && (puck_y - puck_speed <= paddle_y_right + PADDLE_H))) ||
             (((puck_x + PUCK_W != PADDLE_X_RIGHT) && y_dir == 1) && ((puck_y + puck_speed + PUCK_H >= paddle_y_right) && (puck_y + puck_speed <= paddle_y_right + PADDLE_H))) begin

        puck_x <= PADDLE_X_RIGHT - PUCK_W;
        x_dir <= 0; // Bounce Left
    end else begin
        puck_x <= puck_x + puck_speed; // attach to the left side of the right paddle
        game_over <= 1; //bounce left(not losing YET)
        left_right_wins <= 0; // Left player wins
    end
end else begin // Moving Left
    if (puck_x >= puck_speed + PADDLE_X_LEFT + PADDLE_W)
        puck_x <= puck_x - puck_speed;
    else if (((puck_x == PADDLE_X_LEFT + PADDLE_W)  && ((puck_y + PUCK_H >= paddle_y_left) && (puck_y <= paddle_y_left + PADDLE_H)))     ||
             (((puck_x != PADDLE_X_LEFT + PADDLE_W) && y_dir == 0) && ((puck_y - puck_speed + PUCK_H >= paddle_y_left) && (puck_y - puck_speed <= paddle_y_left + PADDLE_H))) ||
             (((puck_x != PADDLE_X_LEFT + PADDLE_W) && y_dir == 1) && ((puck_y + puck_speed + PUCK_H >= paddle_y_left) && (puck_y + puck_speed <= paddle_y_left + PADDLE_H)))   ) begin

        puck_x <= PADDLE_X_LEFT + PADDLE_W; // attach to the right side of the left paddle
        x_dir <= 1; //bounce right(not losing YET)
    end else begin
        puck_x <= puck_x - puck_speed;
        game_over <= 1;
        left_right_wins <= 1; // Right player wins
    end
end
```

The only difference being that now the right side of the puck checks when it hits the left side of the right paddle, whereas the left side of the paddle checks when it hits the right side of the left paddle.

- **Dual VRAM "Winner" Screens:**

To provide an ending to matches, two separate Video RAM (VRAM) instantiations were added to the design. Upon a "Game Over" state, the standard pong graphics are replaced by a pre-loaded image from memory. If the left player wins, a "LEFT WINS" screen is displayed and if the right player wins, a "RIGHT WINS" screen appears.

To make this happen, the “lab3PartB.v” now instantiates 2 vrams, the left_vram and the right_ vram, and the rgb outputs of those vrams are added to the outputs of the “lab3PartB.v”.

The pong module also keeps track of who player wins the round with the left_right_wins register, that takes value in the screenshot shown above in the Multiplayer Mode.

The top module “HDMI_controller_lab4D.v” uses the game_over and the left_right_wins signals and picks which rgb to use for the red, green and blue.

```
wire [7:0] red   = (game_over) ? ((left_right_wins) ? {8{right_rgb[2]}} : {8{left_rgb[2]}}) : red_pong;
wire [7:0] green = (game_over) ? ((left_right_wins) ? {8{right_rgb[1]}} : {8{left_rgb[1]}}) : green_pong;
wire [7:0] blue  = (game_over) ? ((left_right_wins) ? {8{right_rgb[0]}} : {8{left_rgb[0]}}) : blue_pong;
```

- **Scoring System (7-Segment Display):**

Using logic from Lab1, a FourDigitLEDdriver was integrated to track the match score in real-time. The first 7-segment digit displays the left player's score, the second digit displays a separator dash ("-"), and the third digit displays the right player's score.

The scoring system includes a win condition where the first player to reach a score of 3 goals is declared the winner of the match. Once this threshold is reached, a match_over signal is triggered, which resets the game and freezes the final score until a new game is initiated.

The FourDigitLEDdriver which was the top module for Lab1, receives as inputs the game_over and the left_right_wins signals and passes them to the Anode_driver which is the module responsible for activating the anodes and for choosing the char which is passed on to the LEDdecoder, to arrange the cathodes.

Inside the Anode_driver is the message array (4x1 array). The message[0] corresponds to the left player score, message[2] corresponds to the right player score, message[0] corresponds to ("-") and the message[4] is always a value that in the LEDdecoder would mean that the cathodes are off.

When a round is over(game_over) the Anode_driver checks which player won(left_right_wins) and increments their score by one.

```
if (game_over && !prev_game_over) begin
    if (left_right_wins)
        message[2] <= message[2] + 1; // Right player score
    else
        message[0] <= message[0] + 1; // Left player score
end
```

If a player has reached a score of 3 or if the player is about to reach a score of 3 after scoring, the match is over (match_over = 1) and the reloads to ‘0-0’.

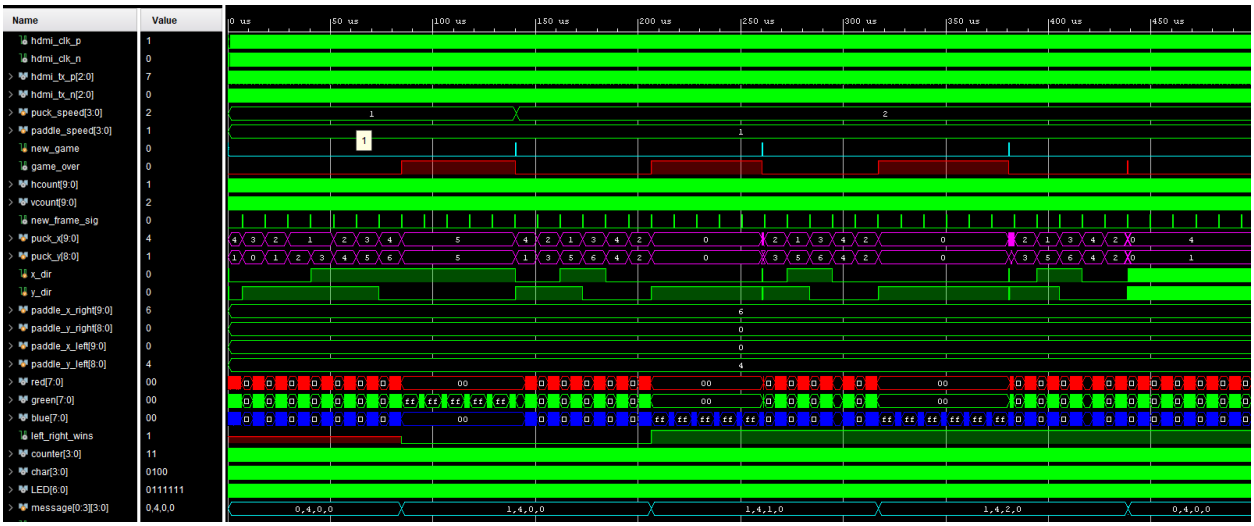
```
// detect match end: any side reached 3
if ((message[0] >= 4'd3) || (message[2] >= 4'd3)) begin
    match_over <= 1'b1;
end else if (game_over && !prev_game_over) begin
    // predict the increment outcome
    if (left_right_wins) begin
        if (message[2] + 1 == 4'd3) match_over <= 1'b1;
    end else begin
        if (message[0] + 1 == 4'd3) match_over <= 1'b1;
    end
end
prev_game_over <= game_over;

// If match is over, display "0 - 0"
if (match_over) begin
    message[0] <= 4'd0;
    message[1] <= 4'b0100; // '-'
    message[2] <= 4'd0;
    message[3] <= 4'd6;
end
```

6.3 Verification

A single waveform shows all 3 bonus ideas working:

The puck coordinates are with the magenta color. The score is kept at the bottom with the aqua color. The new game and the game over are at the top. The left_right_wins is under the rgb in the middle/bottom of the waveform.

NOTE: The puck speed changes from 1 to 2 after the 3rd game. Also, inorder to check in a small simulation which vram’s rgb is used, the first pixels of the left vram have some green pixels, whilste the right vram has some blue pixels. Lastly the message[1] = 4 is the value for the (“-“).



It’s very easy to check that the right paddle makes collisions correctly. The appropriate vram is used when each round is over. Lastly the score is updated accordingly.

6.4 Experiment Proof

Multiplayer Mode 1v1:

Dual VRAM "Winner" Screens:

Note that the vrams using a python script were instantiated wrong, so the LEFT WINS and the RIGHT WINS appeared mirrored in the screen. These doesn't undo the project it still works correctly and provides the appropriate picture according to the situation, there just wasn't enough time in the lab to change the instantiation of the vram manually. So here I mirrored the pictures to be readable.

Scoring System (7-Segment Display):